Yashwanth Giri
Sr. Full Stack Java Developer
Phone: 347-746-5413
Email: giri@techmail2.com
LinkedIn: https://www.linkedin.com/in/yashwanth-reddy-ab6896192/

PROFESSIONAL SUMMARY:

- Java Full Stack Developer with over 10 years of experience delivering enterprise-grade applications across Banking, Healthcare, Finance, Insurance, Telecom, and Retail, leveraging Java 8/11/17/21, Spring Boot, Spring MVC, REST, and SOAP-based architectures.
- Designed secure, modular, and scalable microservices using Spring Cloud (service discovery, API Gateway, centralized config), Spring Security with OAuth2/JWT, and incorporated Spring AOP for logging, exception handling, and transaction management.
- Engineered high-throughput batch jobs and data processing workflows using Spring Batch to automate financial operations such as claims, settlements, and policy reconciliation, improving accuracy and operational efficiency.
- Developed resilient, event-driven systems using Apache Kafka, Kafka Streams, Kafka Connect, RabbitMQ, and IBM MQ to support asynchronous messaging, failover, and distributed coordination between financial modules.
- Delivered cloud-native applications using AWS Lambda, API Gateway, DynamoDB, S3, and Redshift, with built-in logging and secret management via AWS CloudWatch and Secrets Manager.
- Implemented serverless integrations with Salesforce CRM for eligibility updates and real-time claim tracking, ensuring seamless user interactions across systems.
- Built component-driven front ends with React.js, Angular, TypeScript, Redux, and RxJS for dynamic dashboards, CRM workflows, and BI-driven reporting interfaces.
- Upgraded legacy UI stacks using JSP, HTML5, CSS3, Bootstrap, and jQuery, improving performance, accessibility, and usability across multiple platforms.
- Developed GraphQL APIs for optimized, schema-driven data delivery in analytics platforms and internal dashboards, supporting LLM-readiness for NLP integration initiatives.
- Automated infrastructure provisioning with Terraform, Ansible, and CloudFormation, enabling rapid, repeatable deployment of environments across AWS and Azure clouds.
- Built near real-time ingestion and analytics pipelines using Kafka and Azure Functions, processing financial events and telemetry data for monitoring and regulatory reporting.
- Designed and implemented ETL frameworks using StreamSets and Mulesoft for ingestion and enrichment of structured and semi-structured data into Amazon Redshift and other warehouses.
- Created CI/CD pipelines using Jenkins, GitHub Actions, Maven, Gradle, and Bitbucket for secure and traceable deployment, with integrated static analysis and rollback strategies.
- Applied Test-Driven and Behavior-Driven Development using JUnit, Mockito, PowerMock, Cucumber, and Gherkin to enforce code reliability and improve functional test coverage.
- Modeled and managed relational and NoSQL data stores with Oracle, SQL Server, PostgreSQL, MongoDB, and Cassandra, integrating with microservices via JPA, Hibernate, and MyBatis.
- Used Redis caching to enhance performance of high-volume APIs and CRM-integrated services, reducing response latency and improving session stability.
- Enabled observability with centralized logging via ELK Stack, AWS CloudWatch, and Azure Monitor, facilitating proactive support and root cause analysis.
- Configured monitoring dashboards and alerts using Prometheus, Grafana, and Datadog, reducing mean time to resolution (MTTR) and ensuring SLA compliance.
- Published OpenAPI/Swagger documentation for all REST interfaces, simplifying integration with CRM, BI, and frontend platforms.
- Administered Kafka infrastructure using Kafka Admin APIs to ensure optimal partitioning, replication, and cluster health for critical streaming applications.
- Contributed to modernization of legacy systems, including monolithic Java and J2EE services for order management and compliance, enabling phased transition to microservices.
- Followed Agile/Scrum practices including daily standups, sprint planning, retrospectives, and backlog grooming, collaborating with virtual teams across global time zones in highly regulated environments.

TECHNICAL SKILLS

- **Programming Languages:** Java (8/11/17/21), Python, JavaScript, TypeScript, SQL, HTML, CSS
- **Frameworks and Technologies:** Spring Boot, Spring MVC, Spring Batch, Spring Security, Spring Cloud, JPA, Hibernate, My Batis, REST APIs, SOAP Web Services
- **Frontend Libraries:** Angular, React.js, Redux, RxJS, NRx, JSP, HTML5, CSS3, jQuery, JSON
- **Cloud Platforms & Services:** AWS (EC2, EKS, S3, Lambda, RDS, CloudWatch, API Gateway, KMS, SNS, SQS, Secrets Manager, Backup, Redshift), Azure (Azure Kubernetes Service, Blob Storage, Functions, Service Bus, Azure Monitor)
- **Messaging & Integration:** Apache Kafka, Kafka Streams, Kafka Connect, RabbitMQ, IBM MQ, Azure Service Bus
- **DevOps & CI/CD Tools:** Jenkins, GitHub, GitLab, Bitbucket, Docker, Kubernetes, Helm, Terraform, Ansible, CloudFormation
- **Databases:** Oracle, SQL Server, MySQL, PL/SQL, MongoDB, Amazon Redshift
- **Testing & Quality:** JUnit, Mockito, Power Mock, Cucumber, Gherkin, Jest, React Testing Library, Postman, SoapUI, JMeter, Gatling
- **Monitoring & Logging:** AWS CloudWatch, Azure Monitor, ELK Stack, Prometheus, Grafana, Datadog, Log4j
- **Build & Tools:** Maven, Gradle, Git (GitHub, GitLab, Bitbucket), SVN
- **IDEs & Utilities:** IntelliJ IDEA, Eclipse, Spring Tool Suite (STS), MyEclipse, Oracle SQL Developer, IBM RAD, SoapUI.
- **Security & Authentication:** OAuth2.0, JWT, Spring Security
- **Documentation & Collaboration:** Confluence, Jira, UML

WORK EXPERIENCE

Client:Jefferies, New York, NY **August 2025 – Present**
Role: Sr. Java Engineer

Project Description: Designed and delivered a scalable, distributed compute and reporting platform powering a banker-facing financial marketplace solution for revenue attribution and deal analytics. The system supports multi-region workloads, distributed event processing, secure API integrations, and high-performance data aggregation across Finance, Front Office, Compliance, and Audit teams.
The platform was architected using Java 21, Spring Boot microservices, Angular/TypeScript frontend, Kafka event streaming, containerized deployments (Docker/Kubernetes), distributed caching, and AWS cloud infrastructure patterns. Emphasis was placed on resiliency, operational excellence, clean abstractions, distributed compute scaling, and fault-tolerant design.

Responsibilities:

- Designed and implemented distributed microservices forming the core of a banker-facing marketplace and revenue allocation platform supporting Finance, Front Office, Compliance, and Audit teams.
- Architected scalable compute services using event-driven patterns with Kafka and asynchronous messaging to process high-volume financial transactions and allocation workloads.
- Built highly available containerized services using Docker and Kubernetes, enabling horizontal scaling, fault isolation, and resilient multi-instance deployments.
- Collaborated with Node.js and TypeScript teams to define optimal REST abstractions, clean JSON contracts, and reusable middleware patterns following ES2015+ standards.
- Promoted clean architecture principles, modular service design, and maintainable coding practices across distributed Java and Node-based systems.
- Designed and reviewed functional block diagrams, system interaction flows, and data/logic flow charts to communicate architecture decisions with enterprise architects and development teams.
- Evaluated and documented system interfaces, prepared API specifications (OpenAPI), reviewed integration test plans, and validated cross-service communication patterns.
- Implemented AWS-aligned cloud-native architecture patterns including Lambda-triggered workflows (exposure), SQS/SNS-style queue patterns, IAM-based secure access controls, and CloudWatch monitoring integration.
- Contributed to infrastructure-as-code reviews using CloudFormation/SAM templates and supported container lifecycle management via ECR and EKS/ECS deployments.
- Designed strong AWS SQS-style queue consumption patterns including idempotent processing, retry logic,

and dead-letter handling to ensure reliability and resiliency.
- Designed and implemented GraphQL APIs to enable flexible, client-driven data retrieval, reducing over-fetching and improving frontend performance across Angular applications.
- Integrated GraphQL resolvers with Spring Boot microservices and backend services, enabling efficient aggregation of data from multiple distributed sources.
- Engineered distributed caching strategies using Redis/Memcache to reduce database load and optimize compute-heavy aggregation queries.
- Designed and optimized PostgreSQL database schemas for high-volume financial aggregation and reporting workloads, implementing indexing strategies, query tuning, partitioning, and Liquibase-controlled schema migrations to ensure transactional consistency, performance optimization, and controlled production releases across multi-environment deployments.
- Designed and optimized relational (Oracle) and non-relational (MongoDB) schemas, evaluating trade-offs between OLTP, OLAP, and semi-structured audit logging use cases.
- Wrote complex SQL/PLSQL including stored procedures, CTEs, window functions, and aggregation queries to support high-performance financial reporting.
- Owned full SDLC lifecycle including requirements analysis, technical design, estimation, implementation, testing, deployment, and production support within Scrum framework.
- Provided sprint estimates, architectural justifications, and technical trade-off analysis while working closely with product owners and stakeholders.
- Collaborated with DevOps teams to strengthen CI/CD pipelines (Jenkins/Gradle), container publishing, deployment automation, and environment standardization.
- Implemented centralized logging and observability using ELK stack and proactive monitoring to continuously improve operational excellence and system resiliency.
- Led root cause analysis for production issues, performed system debugging across distributed services, and implemented sustainable corrective solutions.
- Secured APIs using Spring Security, SAML, OAuth2, JWT, and RBAC to enforce enterprise-grade authentication, authorization, and compliance requirements.
- Developed well-documented, testable frontend and backend code with strong unit and integration test coverage using JUnit, Mockito, and Cucumber.
- Ensured clean, extensible, and performant code standards across JavaScript/TypeScript and backend services to support long-term maintainability.
- Supported complex system investigations requiring deep analysis of distributed services, database interactions, messaging workflows, and infrastructure behavior.
- Partnered with cross-functional teams to translate regulatory and financial allocation requirements into scalable, resilient distributed platform solutions.

Environment: Java 21, Spring Boot, REST APIs, Microservices, NodeJS (collaboration exposure), TypeScript, ES2015+, Angular 16, JSON, Kafka, AWS (Lambda, SQS, SNS, IAM, CloudWatch, ECR, ECS, EKS), Docker, Kubernetes, Redis, MongoDB, PostgreSQL, GraphQL, Oracle, SQL/PLSQL, CI/CD, Jenkins, Gradle, CloudFormation (exposure), SAM (exposure), ELK Stack, GitLab/Bitbucket, Unix/Linux.

Client: Citigroup, Jersey City, NJ **February 2024 – July 2025**
Role: Sr. Full Stack Java Developer

Project Description: Designed and delivered enterprise-grade Java/J2EE backend services for a real-time compliance and audit platform supporting regulatory frameworks such as CCAR and Basel III. Modernized legacy batch-driven systems into Spring-based RESTful services and event-driven microservices, enabling secure, scalable, and auditable financial data processing. The platform supported regulatory reporting, risk analytics, and investigator workflows, and was built following Agile delivery practices in a highly regulated financial services environment.

Responsibilities:

- Designed and implemented distributed compute services forming the backbone of a real-time compliance and regulatory marketplace platform supporting CCAR and Basel III workloads.
- Transformed legacy batch-based audit systems into scalable, event-driven microservices using Spring Boot and RESTful architecture patterns.
- Engineered distributed processing pipelines using Kafka and messaging-based integrations, enabling asynchronous ingestion of trade events, compliance alerts, and risk signals.
- Collaborated closely with Node.js and TypeScript teams to standardize API contracts, define OpenAPI specifications, and promote ES2015+ best practices across full-stack components.

- Guided development teams in writing clean, extensible, and high-performance JavaScript and backend code aligned with enterprise design standards.
- Designed functional block diagrams, system interaction flows, and data/logic flow charts to communicate architecture decisions with enterprise architects and cross-team stakeholders.
- Reviewed and validated system interfaces, API documentation, test plans, and integration specifications to ensure regulatory-grade traceability.
- Built cloud-aligned microservices supporting AWS-based deployment strategies, including Lambda-triggered processing (exposure), SQS-style queue patterns, IAM policy integration, and CloudWatch monitoring.
- Contributed to CloudFormation and infrastructure-as-code governance reviews to support controlled multi-environment deployments.
- Implemented robust AWS SQS queue consumption patterns including retry logic, dead-letter queue handling, and idempotent message processing for resiliency.
- Designed containerized services using Docker with layered image strategies and deployed across Kubernetes clusters for fault-tolerant, horizontally scalable distributed systems.
- Integrated CI/CD pipelines using Jenkins and Gradle, supporting automated testing, container publishing (ECR-style registries), and controlled environment promotions.
- Developed high-throughput backend services with advanced JVM tuning, garbage collection analysis, thread profiling, and heap optimization to support large-scale regulatory investigations.
- Designed relational (Oracle) and semi-structured (audit metadata) data models, evaluating trade-offs between transactional and analytical workloads.
- Implemented secure GraphQL endpoints with JWT, OAuth2, and RBAC-based field-level authorization to enforce fine-grained access controls for sensitive financial data.
- Collaborated with Angular/TypeScript teams to transition selective use cases from REST to GraphQL-based data consumption, reducing API round trips and improving UI responsiveness.
- Designed and optimized PostgreSQL schemas for regulatory reporting and audit snapshot workloads, implementing indexing strategies, query plan analysis, partitioning, and Liquibase-driven schema migrations to support high-volume compliance data processing while maintaining ACID consistency and performance across multi-environment deployments.
- Wrote advanced SQL and PL/SQL including complex joins, aggregations, reconciliation logic, window functions, and regulatory snapshot generation queries.
- Built caching layers using Redis and Memcache to optimize frequently accessed compliance and audit datasets in distributed environments.
- Supported database deployment, configuration, monitoring, and debugging activities across multiple environments.
- Implemented secure REST APIs using Spring Security, SAML SSO, JWT, and RBAC, ensuring enterprise IAM alignment and regulatory-grade access controls.
- Enabled distributed monitoring and observability using ELK stack, Prometheus, and Grafana to continuously improve operational excellence and resiliency.
- Performed root cause investigations for complex system issues spanning microservices, messaging brokers, and database layers, recommending sustainable corrective actions.
- Collaborated with DevOps to enhance container orchestration practices (ECS/EKS exposure), deployment pipelines, and cloud cost optimization strategies.
- Delivered well-documented, testable frontend and backend code following TDD and BDD methodologies using JUnit and Cucumber.
- Worked with stakeholders to evaluate regulatory requirements and convert them into actionable technical work items within Scrum ceremonies.
- Provided engineering support for large-scale, multi-system integrations requiring deep analysis of distributed system design specifications and cross-platform dependencies.
- Promoted event-driven and microservice-based architecture patterns, polyglot programming exposure (Java, Node.js, TypeScript), and secure API-first development practices.

Environment: Java 11/17, J2EE, Spring Boot, Spring Framework, Spring MVC, Spring Security, REST APIs, Microservices, Node.js (collaboration exposure), TypeScript (ES2015+), Angular / AngularJS, HTML5, CSS3, JSON, Kafka, GraphQL, AWS (Lambda, SQS, SNS, IAM, CloudWatch, CloudTrail, ECR, ECS/EKS – exposure), Docker, Kubernetes, Oracle Database, PostgreSQL, SQL, PL/SQL, Redis, Memcache, Elastic Stack (ELK), Prometheus, Grafana, Jenkins, Gradle, CloudFormation (exposure), SAM (exposure), Swagger/OpenAPI, WebLogic, Unix/Linux, Shell Scripting, GitLab/Bitbucket, CI/CD.

Client: CVS Health, Austin, TX **June 2022 – November 2023**
Role: Full Stack Developer

Project Description: Developed a secure, enterprise-grade Java/J2EE microservices platform for healthcare claims and eligibility processing, supporting regulatory, audit, and financial reconciliation workflows. The platform exposed RESTful APIs, integrated with CRM and BI systems, processed high-volume event streams, and followed HIPAA and enterprise security standards. Built using Spring Framework, Angular/TypeScript, SQL-based databases, and containerized deployments, the solution emphasized performance, reliability, and automated delivery within an Agile, cross-functional environment.

Responsibilities:

- Designed and implemented distributed healthcare claims processing services forming part of a scalable enterprise platform supporting eligibility validation, financial reconciliation, and regulatory compliance workflows.
- Built production-grade Node.js–compatible REST integrations and TypeScript-based service abstractions, promoting ES2015+ standards and clean API-first architecture patterns.
- Architected event-driven microservices using Kafka and RabbitMQ to process high-volume claims events, retries, and downstream synchronization across distributed systems.
- Developed AWS-aligned cloud-native components leveraging Lambda-triggered processing (exposure), SQS queue design patterns, SNS notifications, IAM policies, CloudWatch monitoring, and CloudTrail audit logging.
- Designed resilient multi-AZ deployment strategies using Docker containers and Kubernetes orchestration to ensure high availability and controlled rollouts.
- Promoted microservice-based and polyglot programming principles, enabling collaboration across Java, Node.js, and TypeScript components within a distributed compute environment.
- Designed and reviewed functional block diagrams, data flow charts, and integration specifications to communicate system behavior with architecture and DevOps teams.
- Implemented strong AWS SQS queue consumption patterns with idempotency, retry mechanisms, dead-letter queues, and monitoring for reliability and resiliency.
- Built containerized services with optimized Docker layering strategies and integrated deployments through CI/CD pipelines using Jenkins and Gradle.
- Supported infrastructure-as-code practices with exposure to CloudFormation/SAM templates and configuration-driven environment provisioning.
- Designed relational database schemas and optimized SQL queries (CTEs, indexed views, stored procedures) to support high-throughput transactional and reconciliation workloads.
- Evaluated trade-offs between relational and non-relational data stores, integrating Redis caching and in-memory caching for performance optimization.
- Implemented secure API development using Spring Security, OAuth2, JWT, and SAML-style authentication aligned with HIPAA and enterprise IAM standards.
- Built Angular/TypeScript front-end modules consuming REST OpenAPI-documented services, ensuring clean separation of concerns and maintainable UI architecture.
- Developed distributed data ingestion pipelines to process batch and real-time healthcare feeds into analytical systems for BI reporting and audits.
- Implemented centralized logging and observability using ELK stack, with proactive monitoring dashboards via Prometheus and Grafana to improve operational excellence.
- Partnered with DevOps to strengthen container orchestration workflows using ECS/EKS exposure and improve deployment automation and cloud cost awareness.
- Worked closely with internal stakeholders to evaluate healthcare compliance requirements and translate them into technical work items within Scrum ceremonies.
- Provided production support and complex system troubleshooting across microservices, messaging brokers, databases, and container infrastructure.
- Designed and managed PostgreSQL databases for healthcare claims and eligibility processing, implementing normalization strategies, indexing, query performance tuning, and Liquibase-based schema versioning to support high-volume transactional workloads while maintaining HIPAA-compliant data integrity and audit traceability across environments.
- Promoted clean, extensible, and testable code practices in JavaScript and backend services, enforcing modular design patterns for long-term maintainability.
- Applied TDD/BDD methodologies using JUnit, Mockito, and Cucumber to ensure reliability of claims validation and eligibility business rules.
- Ensured compliance with healthcare security standards while supporting integration with external payment gateways and claims settlement systems.

- Maintained versioned REST contracts using Swagger/OpenAPI, enabling backward compatibility and smooth integration with partner systems.
- Delivered accurate effort estimates and technical oversight within Agile Scrum environments, contributing to sprint planning and release management.

Environment: Java 8/11, J2EE, Spring Framework, Spring MVC, REST APIs, Microservices, Node.js (collaboration exposure), TypeScript (ES2015+), Angular / AngularJS, HTML5, CSS3, JSON, Kafka, RabbitMQ, AWS (Lambda, SQS, SNS, IAM, CloudWatch, CloudTrail, ECS/EKS – exposure), Docker, Kubernetes, Redis, Memcache, Relational Databases (SQL, PL/SQL), PostgreSQL, Swagger/OpenAPI, Jenkins, Gradle, CloudFormation (exposure), SAM (exposure), ELK Stack, Prometheus, Grafana, Unix/Linux, Shell Scripting, OAuth2, JWT, SAML, CI/CD, GitLab/Bitbucket.

Client: GEICO, Kansas City, MO **November 2019 – May 2022**

Role: Sr. Java Developer

Project Description: Delivered enterprise Java/J2EE backend systems supporting policy administration, claims processing, and payment workflows within a highly regulated financial services environment. Modernized legacy systems into Spring-based RESTful services and event-driven components, enabling scalable integration with CRM, BI, and reporting platforms. The solution emphasized performance, security, auditability, and automated delivery, supporting distributed Agile teams and strict regulatory SLAs.

Responsibilities:

- Engineered distributed backend services supporting policy administration, claims lifecycle management, and payment processing within a high-availability enterprise platform.
- Re-architected legacy monolithic components into microservice-based and event-driven systems to improve scalability, resiliency, and deployment agility.
- Designed compute-intensive services handling policy renewals, endorsements, claim adjudication, and payment settlement workflows in a decoupled architecture.
- Implemented RESTful APIs following OpenAPI standards, enabling seamless integration with CRM, BI, and Angular-based UI platforms.
- Built asynchronous messaging pipelines using Kafka to process policy change events, claim updates, payment notifications, and escalation triggers.
- Designed reliable queue-based processing patterns inspired by AWS SQS-style consumption models with retry handling, dead-letter management, and idempotent consumers.
- Containerized backend services using Docker and supported orchestration strategies aligned with Kubernetes-based deployments for horizontal scaling.
- Partnered with DevOps to automate build and release pipelines using Jenkins, enabling CI/CD-driven deployments and environment promotions.
- Developed Spring Batch workflows to process high-volume nightly reconciliation, billing exports, and structured reporting datasets.
- Designed relational database schemas and optimized SQL/PLSQL queries using analytic functions, aggregation views, and indexing strategies to support high-throughput transactional systems.
- Modeled historical policy and claims snapshots across relational and NoSQL stores to ensure audit traceability and regulatory compliance.
- Implemented distributed caching strategies using Redis and in-memory caches to improve response times for eligibility validation and premium calculations.
- Secured APIs using Spring Security, OAuth2, SAML-style integrations, and RBAC, enforcing fine-grained access control across policy administrators and claims processors.
- Built fault-tolerant stateless services for document generation, outbound notifications, and third-party integrations with payment gateways.
- Collaborated with frontend teams using Angular and TypeScript to ensure clean JSON contracts and maintainable service abstractions.
- Produced functional block diagrams, UML models, and system flow documentation to support architecture governance and cross-team collaboration.
- Monitored JVM performance, API latency, and messaging throughput using Grafana and centralized logging tools to reduce production incidents and improve reliability.
- Supported large-scale data ingestion and transformation pipelines feeding BI platforms for actuarial analysis and claims trend reporting.
- Participated in Agile Scrum ceremonies, providing estimates, technical trade-off analysis, and sprint deliverables across distributed teams.

- Conducted root cause analysis for production issues spanning APIs, databases, and messaging brokers, delivering sustainable corrective actions.
- Maintained versioned REST contracts and integration documentation to support long-term extensibility and backward compatibility.
- Promoted clean code standards, modular design patterns, and testable service architecture aligned with enterprise development best practices.

Environment: Java 8, J2EE, Spring Framework, Spring MVC, Spring Batch, REST APIs, SOAP, Microservices, Angular, TypeScript, HTML5, CSS3, JSON, Kafka, Redis, Memcache, Relational Databases (SQL, PL/SQL), NoSQL (snapshot modeling), Docker, Kubernetes (exposure), Jenkins, CI/CD, Swagger/OpenAPI, OAuth2, SAML, RBAC, JUnit, Postman, Grafana, ELK Stack (exposure), Unix/Linux, Shell Scripting, Git.

Client: Comcast, Denver, CO **October 2017 – October 2019**
Role: Java Developer

Project Description: Developed and maintained REST/SOAP microservices for customer account management, billing, and content delivery using Java 8 and Spring Boot. Automated high-volume workflows with Spring Batch and ensured service availability using Spring Cloud. Managed data with Oracle, My Batis, and MongoDB for personalized content and accurate invoicing. Containerized services with Docker, deployed via Terraform and Ansible, and implemented CI/CD with Jenkins and Git to streamline deployments across cloud environments.

Responsibilities:

- Developed and maintained RESTful and SOAP-based microservices using Java 8, Spring Boot, and Spring MVC to support Comcast's customer account management, billing, and content delivery platforms across web, mobile, and set-top box systems.
- Automated backend workflows with Spring Batch to handle high-volume operations such as monthly billing cycles, service activations, subscription renewals, and outage notifications, reducing manual interventions and improving throughput.
- Implemented Spring Cloud (Brixton) for distributed configuration, service discovery, and fault tolerance to ensure high availability of microservices like channel lineup services, device provisioning, and support ticketing.
- Managed structured customer and subscription data using Hibernate with Oracle 12c, and performed complex billing rule calculations via MyBatis, ensuring accurate invoicing and compliance with regional tax regulations.
- Integrated MongoDB to persist user preferences, recently viewed content, and interaction logs, enhancing personalized content recommendations and targeted ads on Comcast's Xfinity platforms.
- Wrote unit and integration tests using JUnit, Mockito, and PowerMock, reducing regression issues during feature rollouts and ensuring application stability during high-traffic TV events and promotions.
- Containerized microservices using Docker, and provisioned environments using Terraform and Ansible, enabling consistent deployments across development, QA, and production.
- Conducted performance and load testing using JMeter, identifying bottlenecks in video-on-demand (VoD) APIs and optimizing service latency under peak usage.
- Built CI/CD pipelines with Jenkins and Git, automating build, test, and deployment workflows for Comcast's cloud-hosted microservices.
- Documented service contracts, exception handling strategies, and integration guides using Confluence, improving cross-team collaboration between development, QA, and DevOps.

Environment: Java 8, Spring Boot, Spring MVC, Spring Batch, Spring Cloud (Brixton), Oracle 12c, MongoDB, Hibernate, My Batis, Docker, Terraform, Ansible, Jenkins, Git, JUnit, Mockito, Power Mock, JMeter, Confluence.

Client: Fairway, Dallas, TX **March 2016 – September 2017**
Role: Java Developer

Project Description: Maintained and modernized legacy J2EE modules for order processing, pricing, and inventory using Struts, JSP, Servlets, and EJB. Refactored monolithic components into modular services and integrated backend systems with IBM MQ and Oracle 10g. Developed Java Swing tools for store operations and processed vendor feeds using JAXB. Deployed applications on Tomcat and WebLogic, ensuring stability and compatibility across retail workflows.

Responsibilities:

- Maintained and enhanced legacy modules built on Java 6, J2EE, and frameworks like Struts 1.3, JSP 2.0, Servlets 2.5, and EJB 2.1, supporting core workflows such as order processing, promotional pricing, and store inventory sync.
- Refactored tightly coupled J2EE components into modular services, improving code maintainability and allowing smoother implementation of catalog changes and seasonal promotions.
- Integrated backend retail systems with mainframe processes using IBM MQ 7.5 and flat file batch transfers, ensuring reliable nightly inventory reconciliation across distributed store networks.
- Wrote and optimized PL/SQL procedures in Oracle 10g for applying dynamic discounts, tax calculations, and validating cart-level constraints during checkout.
- Built desktop-based merchandising tools using Java Swing for managing product entries, barcode printing, and location-based price overrides, improving efficiency for store managers.
- Deployed WAR files to Apache Tomcat 6.0 and Oracle WebLogic Server 10.3, handled class path issues and server tuning to ensure compatibility with legacy enterprise.

Environment: Java 6, J2EE, Struts 1.3, JSP 2.0, Servlets 2.5, EJB 2.1, Oracle 10g, PL/SQL, Java Swing, IBM MQ 7.5, Apache Tomcat 6.0, Oracle WebLogic 10.3, Log4j 1.2, XML.

Client: Bath & Body Works, Cincinnati, OH **January 2015 – February 2016**
Role: Junior Java Developer

Project Description: Developed Spring Boot microservices for catalog, inventory, and order features on the e-commerce platform. Integrated RESTful APIs with frontend teams for cart and user management, and supported Agile delivery with unit testing, documentation, and CI/CD using Git, Jenkins, and Jira.

Responsibilities:

- Developed and enhanced backend services using Java 8, Spring Boot, and RESTful APIs to support key features like product catalog, inventory lookup, and order placement on the e-commerce platform.
- Collaborated with frontend teams to integrate backend APIs for cart management, user authentication, and promotional offers, ensuring smooth cross-platform performance.
- Supported quality assurance by writing unit and integration tests using JUnit and Mockito, and participated in Agile ceremonies for sprint planning, code reviews, and deployments via Git, Jenkins, and Jira.

Environment: Java 8, Spring Boot, Spring MVC, RESTful APIs, JUnit, Mockito, Git, Jenkins, Jira, Confluence, MySQL, Postman, IntelliJ IDEA, Agile/Scrum.

EDUCATION: Bachelors in Computer Science, University of Cincinnati (Jan 2011 - Dec 2014)